

Lazy Loading in JavaScript - Comprehensive Notes

Table of Contents

1. Introduction & Core Concepts
 2. Why Lazy Loading?
 3. Types of Lazy Loading
 4. Implementation Methods
 5. Interview Questions & Answers
 6. Real-World Examples
 7. Best Practices
 8. Performance Metrics
 9. Common Pitfalls
-

1. Introduction & Core Concepts {#introduction}

What is Lazy Loading?

Lazy loading is a design pattern and optimization technique where resources (images, scripts, components, data) are loaded only when they are needed, rather than loading everything upfront during initial page load.

Key Principle: "Don't load what you don't need right now"

Core Benefits

- Reduces initial page load time
 - Decreases bandwidth consumption
 - Improves Time to Interactive (TTI)
 - Better user experience on slow networks
 - Reduces server load
-

2. Why Lazy Loading? {#why-lazy-loading}

Performance Metrics Impact

Before Lazy Loading:

Initial Load: 5MB (all resources)
Time to Interactive: 8 seconds
First Contentful Paint: 3 seconds

After Lazy Loading:

Initial Load: 1.5MB (critical resources only)
Time to Interactive: 2.5 seconds
First Contentful Paint: 1 second

Use Cases

1. **Image-heavy websites** (e-commerce, portfolios, galleries)
 2. **Single Page Applications** (code splitting for routes)
 3. **Infinite scroll** implementations
 4. **Large data tables** (virtual scrolling)
 5. **Component libraries** (load on demand)
 6. **Video streaming platforms**
 7. **Social media feeds**
-

3. Types of Lazy Loading {#types}

A. Image Lazy Loading

Native Lazy Loading (HTML)

```
<!-- Modern browsers support native lazy loading -->  
  
  
<!-- Attributes -->  
loading="lazy"      <!-- Load when near viewport -->  
loading="eager"     <!-- Load immediately (default) -->  
loading="auto"      <!-- Browser decides -->
```

Intersection Observer API

```
// More control and customization
const imageObserver = new IntersectionObserver((entries, observer) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
      img.classList.remove('lazy');
      observer.unobserve(img);
    }
  });
});

// Observe all lazy images
document.querySelectorAll('img[data-src]').forEach(img => {
  imageObserver.observe(img);
});
```

HTML Structure:

```

```

B. Component/Module Lazy Loading

Dynamic Import (ES6+)

```
// Instead of static import
// import HeavyComponent from './HeavyComponent';

// Use dynamic import
button.addEventListener('click', async () => {
  const module = await import('./HeavyComponent.js');
  const component = module.default;
  component.render();
});
```

React Lazy Loading

```
import React, { lazy, Suspense } from 'react';

// Lazy load component
const HeavyComponent = lazy(() => import('./HeavyComponent'));

function App() {
  return (
    <div>
      <Suspense fallback=<div>Loading...</div>>
        <HeavyComponent />
      </Suspense>
    </div>
  );
}
```

Vue Lazy Loading

```
// Route-based code splitting
const routes = [
  {
    path: '/dashboard',
    component: () => import('./views/Dashboard.vue')
  }
];
```

C. Script Lazy Loading

```
function loadScript(src) {
  return new Promise((resolve, reject) => {
    const script = document.createElement('script');
    script.src = src;
    script.async = true;
    script.onload = () => resolve(script);
    script.onerror = () => reject(new Error(`Script load error: ${src}`));
    document.head.appendChild(script);
  });
}

// Usage
loadScript('https://cdn.example.com/library.js')
  .then(() => console.log('Script loaded!'))
  .catch(err => console.error(err));
```

D. Data/Content Lazy Loading (Infinite Scroll)

```
class InfiniteScroll {
  constructor(container, fetchDataCallback) {
    this.container = container;
    this.fetchData = fetchDataCallback;
    this.page = 1;
    this.loading = false;
    this.init();
  }

  init() {
    window.addEventListener('scroll', () => this.handleScroll());
  }

  handleScroll() {
    const scrollTop = window.scrollY + window.innerHeight;
    const threshold = document.documentElement.scrollHeight - 200;

    if (scrollTop >= threshold && !this.loading) {
      this.loadMore();
    }
  }

  async loadMore() {
    this.loading = true;
    try {
      const data = await this.fetchData(this.page);
      this.renderData(data);
      this.page++;
    } catch (error) {
      console.error('Error loading data:', error);
    } finally {
      this.loading = false;
    }
  }

  renderData(data) {
    data.forEach(item => {
      const element = document.createElement('div');
      element.textContent = item.content;
      this.container.appendChild(element);
    });
  }
}

// Usage
const scroll = new InfiniteScroll(
  document.getElementById('content'),
  async (page) => {
    const response = await fetch(`/api/posts?page=${page}`);
    return response.json();
  }
);
```

4. Implementation Methods {#implementation}

Method 1: Intersection Observer (Recommended)

Advantages:

- Better performance than scroll events
- Native browser API
- Configurable thresholds
- Automatic cleanup

```
const lazyLoad = (selector, options = {}) => {
  const defaultOptions = {
    root: null,
    rootMargin: '50px',
    threshold: 0.01
  };

  const config = { ...defaultOptions, ...options };

  const observer = new IntersectionObserver((entries, self) => {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        const element = entry.target;

        // For images
        if (element.tagName === 'IMG') {
          element.src = element.dataset.src;
          if (element.dataset.srcset) {
            element.srcset = element.dataset.srcset;
          }
        }

        // For iframes
        if (element.tagName === 'IFRAME') {
          element.src = element.dataset.src;
        }

        // Remove lazy class and stop observing
        element.classList.remove('lazy');
        self.unobserve(element);
      }
    });
  }, config);

  // Observe all elements matching selector
  document.querySelectorAll(selector).forEach(el => observer.observe(el));

  return observer;
};

// Usage
lazyLoad('.lazy');
```

Method 2: Scroll Event (Legacy Support)

```
function lazyLoadWithScroll() {
  const lazyImages = document.querySelectorAll('.lazy');

  function loadImage() {
    lazyImages.forEach(img => {
      if (img.getBoundingClientRect().top < window.innerHeight + 200) {
        img.src = img.dataset.src;
        img.classList.remove('lazy');
      }
    });
  }

  // Throttle scroll event
  let ticking = false;
  window.addEventListener('scroll', () => {
    if (!ticking) {
      window.requestAnimationFrame(() => {
        loadImage();
        ticking = false;
      });
      ticking = true;
    }
  });

  // Initial load
  loadImage();
}
```

Method 3: Progressive Image Loading (Blur-up)

```

class ProgressiveImage {
  constructor(container) {
    this.container = container;
    this.placeholder = container.querySelector('.placeholder');
    this.fullImage = new Image();
  }

  load(src) {
    this.fullImage.src = src;
    this.fullImage.onload = () => {
      this.reveal();
    };
  }

  reveal() {
    this.fullImage.classList.add('loaded');
    this.container.appendChild(this.fullImage);
    setTimeout(() => {
      this.placeholder.remove();
    }, 300);
  }
}

// CSS
/*
.placeholder {
  filter: blur(10px);
  transform: scale(1.1);
}

.loaded {
  animation: fadeIn 0.3s ease-in;
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
*/

```


Method 4: Webpack Code Splitting

```
// webpack.config.js
module.exports = {
  optimization: {
    splitChunks: {
      chunks: 'all',
      cacheGroups: {
        vendor: {
          test: /[\\/]node_modules[\\/]/,
          name: 'vendors',
          priority: 10
        }
      }
    }
  }
};

// In your code
import(/* webpackChunkName: "chart" */ './chart.js')
  .then(module => {
    module.renderChart();
  });
```

5. Interview Questions & Answers {#interview-questions}

Q1: What is lazy loading and why is it important?

Answer: Lazy loading is an optimization technique that defers loading of non-critical resources until they're needed. It's important because:

- Reduces initial bundle size and load time
- Saves bandwidth for users on limited data plans
- Improves Core Web Vitals (LCP, FID, CLS)
- Enhances user experience on slower networks
- Reduces server costs by serving less data initially

Q2: Explain the difference between eager loading and lazy loading?

Answer:

- **Eager Loading:** All resources load immediately when page loads
 - Pros: Everything available instantly, no loading delays
 - Cons: Slower initial load, wastes bandwidth
- **Lazy Loading:** Resources load only when needed
 - Pros: Faster initial load, efficient bandwidth usage
 - Cons: Slight delay when accessing lazy-loaded content

Q3: What is Intersection Observer and how does it work?

Answer: Intersection Observer is a browser API that asynchronously observes changes in the intersection of a target element with viewport or parent element.

Key concepts:

```
const observer = new IntersectionObserver(callback, options);
// callback: Function called when intersection changes
// options: { root, rootMargin, threshold }
```

- **root:** Element to check intersection against (default: viewport)
- **rootMargin:** Margin around root (like CSS margin)
- **threshold:** Percentage of visibility to trigger callback (0-1)

Advantages over scroll events:

- Doesn't run on main thread
- Better performance
- No need for throttling/debouncing
- More accurate

Q4: How would you implement image lazy loading?

Answer: Three approaches:

1. Native HTML:

```

```

2. Intersection Observer:

```
const observer = new IntersectionObserver(entries => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.src = entry.target.dataset.src;
      observer.unobserve(entry.target);
    }
  });
});
document.querySelectorAll('[data-src]').forEach(img => observer.observe(img));
```

3. Library (like lazysizes):

```

```

Q5: What are the challenges with lazy loading?

Answer:

1. **SEO concerns:** Search engines may not execute JavaScript
 - Solution: Use server-side rendering or provide fallbacks
2. **Layout shifts:** Content jumping when images load
 - Solution: Define explicit width/height, use aspect ratio boxes
3. **JavaScript dependency:** Won't work if JS disabled
 - Solution: Use `<noscript>` tags for fallbacks
4. **Initial viewport images:** Should NOT be lazy loaded
 - Solution: Load above-the-fold content immediately
5. **Browser support:** Older browsers don't support new APIs
 - Solution: Polyfills or fallback methods

Q6: How do you handle lazy loading with SEO?

Answer:

```
<!-- Combine approaches for SEO -->


<!-- Use noscript fallback -->
<noscript>
  
</noscript>
```

Also consider:

- Implement server-side rendering (SSR)
- Use structured data markup
- Ensure proper alt attributes
- Consider prerendering for crawlers

Q7: Explain code splitting and its relation to lazy loading?

Answer: Code splitting divides your JavaScript bundle into smaller chunks that can be loaded on demand.

Types:

1. **Route-based:** Load code for each route separately

```
const Home = lazy(() => import('./Home'));  
const About = lazy(() => import('./About'));
```

2. **Component-based**: Load heavy components when needed

```
const HeavyChart = lazy(() => import('./Chart'));
```

3. **Vendor splitting**: Separate third-party libraries

```
// webpack automatically splits node_modules
```

Q8: What is the "loading" attribute in images?

Answer: Native browser feature for lazy loading:

```

```

Values:

- **lazy**: Load when near viewport
- **eager**: Load immediately (default)
- **auto**: Browser decides

Browser support: Modern browsers (Chrome 76+, Firefox 75+, Safari 15.4+)

Q9: How do you measure the impact of lazy loading?

Answer: Key metrics to track:

1. **Load Time Metrics**:

- First Contentful Paint (FCP)
- Largest Contentful Paint (LCP)
- Time to Interactive (TTI)

2. **Network Metrics**:

- Initial bundle size
- Total bytes transferred
- Number of requests

3. **Tools**:

```
// Performance API
performance.mark('lazyLoadStart');
// ... lazy load code
performance.mark('lazyLoadEnd');
performance.measure('lazyLoad', 'lazyLoadStart', 'lazyLoadEnd');
```

Q10: What's the difference between prefetch and preload?

Answer:

```
<!-- Preload: High priority, fetch ASAP -->
<link rel="preload" href="critical.css" as="style">

<!-- Prefetch: Low priority, fetch when idle -->
<link rel="prefetch" href="next-page.js">
```

- **Preload:** For current page critical resources
 - **Prefetch:** For future navigation resources
 - Both work with lazy loading strategy
-

6. Real-World Examples {#examples}

Example 1: E-commerce Product Gallery

```
class ProductGallery {
  constructor(container) {
    this.container = container;
    this.observer = new IntersectionObserver(
      this.handleIntersection.bind(this),
      { rootMargin: '100px' }
    );
    this.init();
  }

  init() {
    const products = this.container.querySelectorAll('.product-card');
    products.forEach(product => this.observer.observe(product));
  }

  handleIntersection(entries) {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        this.loadProduct(entry.target);
        this.observer.unobserve(entry.target);
      }
    });
  }

  loadProduct(element) {
    const img = element.querySelector('img[data-src]');
    const priceElement = element.querySelector('[data-price-url]');

    // Load image
    if (img) {
      img.src = img.dataset.src;
      img.onload = () => img.classList.add('loaded');
    }

    // Fetch dynamic price
    if (priceElement) {
      fetch(priceElement.dataset.priceUrl)
        .then(res => res.json())
        .then(data => {
          priceElement.textContent = `$$${data.price}`;
        });
    }
  }
}

// Usage
new ProductGallery(document.querySelector('.product-gallery'));
```

Example 2: Social Media Feed with Infinite Scroll

```

class SocialFeed {
  constructor(feedContainer) {
    this.container = feedContainer;
    this.page = 1;
    this.loading = false;
    this.hasMore = true;
    this.setupScrollListener();
  }

  setupScrollListener() {
    const sentinel = document.createElement('div');
    sentinel.className = 'scroll-sentinel';
    this.container.appendChild(sentinel);

    const observer = new IntersectionObserver(
      entries => {
        if (entries[0].isIntersecting && !this.loading && this.hasMore) {
          this.loadMore();
        }
      },
      { rootMargin: '300px' }
    );

    observer.observe(sentinel);
  }

  async loadMore() {
    this.loading = true;
    this.showLoader();

    try {
      const response = await fetch(`/api/feed?page=${this.page}`);
      const data = await response.json();

      if (data.posts.length === 0) {
        this.hasMore = false;
        this.showEndMessage();
        return;
      }

      this.renderPosts(data.posts);
      this.page++;
    } catch (error) {
      this.showError(error);
    } finally {
      this.loading = false;
      this.hideLoader();
    }
  }

  renderPosts(posts) {
    const fragment = document.createDocumentFragment();

    posts.forEach(post => {
      const postElement = this.createPostElement(post);
      fragment.appendChild(postElement);
    });

    this.container.insertBefore(
      fragment,
      this.container.querySelector('.scroll-sentinel')
    );
  }
}

```

```

createPostElement(post) {
  const article = document.createElement('article');
  article.className = 'post';
  article.innerHTML = `
    
    <h3>${post.title}</h3>
    <p>${post.excerpt}</p>
  `;

  // Lazy load image
  const img = article.querySelector('img');
  const imgObserver = new IntersectionObserver(entries => {
    if (entries[0].isIntersecting) {
      img.src = img.dataset.src;
      imgObserver.unobserve(img);
    }
  });
  imgObserver.observe(img);

  return article;
}

showLoader() {
  const loader = document.createElement('div');
  loader.className = 'loading-spinner';
  loader.innerHTML = 'Loading...';
  this.container.appendChild(loader);
}

hideLoader() {
  const loader = this.container.querySelector('.loading-spinner');
  if (loader) loader.remove();
}

showEndMessage() {
  const message = document.createElement('div');
  message.className = 'end-message';
  message.textContent = "You've reached the end!";
  this.container.appendChild(message);
}

showError(error) {
  console.error('Error loading posts:', error);
  const errorMsg = document.createElement('div');
  errorMsg.className = 'error-message';
  errorMsg.textContent = 'Failed to load posts. Please try again.';
  this.container.appendChild(errorMsg);
}

// Initialize
new SocialFeed(document.getElementById('feed'));

```


Example 3: Video Lazy Loading

```

class LazyVideo {
  constructor() {
    this.videos = document.querySelectorAll('video[data-src]');
    this.observer = new IntersectionObserver(
      this.handleIntersection.bind(this),
      { threshold: 0.25 }
    );
    this.init();
  }

  init() {
    this.videos.forEach(video => this.observer.observe(video));
  }

  handleIntersection(entries) {
    entries.forEach(entry => {
      if (entry.isIntersecting) {
        this.loadVideo(entry.target);
      } else {
        this.pauseVideo(entry.target);
      }
    });
  }

  loadVideo(video) {
    if (!video.src) {
      video.src = video.dataset.src;
      video.load();
    }

    video.play().catch(error => {
      console.log('Autoplay prevented:', error);
    });
  }

  pauseVideo(video) {
    if (!video.paused) {
      video.pause();
    }
  }
}

// Usage
new LazyVideo();

```

7. Best Practices {#best-practices}

1. Always Set Dimensions

```
<!-- ❌ Bad: No dimensions -->


<!-- ✅ Good: Explicit dimensions -->


<!-- ✅ Better: Aspect ratio container -->
<div class="aspect-ratio-box" style="aspect-ratio: 16/9;">
  
</div>
```

2. Use Low-Quality Placeholders

```
<!-- LQIP (Low Quality Image Placeholder) -->

```

3. Prioritize Above-the-Fold Content

```
// Don't lazy load images in first viewport
const isInInitialViewport = (element) => {
  const rect = element.getBoundingClientRect();
  return rect.top < window.innerHeight;
};

images.forEach(img => {
  if (isInInitialViewport(img)) {
    img.src = img.dataset.src; // Load immediately
  } else {
    observer.observe(img); // Lazy load
  }
});
```

4. Provide Loading Indicators

```
<div class="image-container">
  <div class="skeleton-loader"></div>
  
</div>
```

```

.skeleton-loader {
  width: 100%;
  height: 100%;
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 200% 100%;
  animation: loading 1.5s infinite;
}

@keyframes loading {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}

```

5. Handle Errors Gracefully

```

img.addEventListener('error', function() {
  this.src = 'fallback-image.jpg';
  this.classList.add('error');
});

```

6. Use Appropriate Root Margins

```

// Load slightly before entering viewport
const observer = new IntersectionObserver(callback, {
  rootMargin: '50px 0px' // Load 50px before visible
});

```

7. Implement Retry Logic

```

async function loadWithRetry(url, retries = 3) {
  for (let i = 0; i < retries; i++) {
    try {
      const response = await fetch(url);
      if (response.ok) return response;
    } catch (error) {
      if (i === retries - 1) throw error;
      await new Promise(resolve => setTimeout(resolve, 1000 * Math.pow(2, i)));
    }
  }
}

```

8. Optimize for Different Network Conditions

```
// Check network speed
const connection = navigator.connection || navigator.mozConnection || navigator.webkitC
onnection;

if (connection) {
  const effectiveType = connection.effectiveType;

  if (effectiveType === '4g') {
    // Aggressive lazy loading
    rootMargin = '200px';
  } else if (effectiveType === '3g') {
    // Conservative lazy loading
    rootMargin = '50px';
  } else {
    // Very conservative on slow connections
    rootMargin = '0px';
  }
}
```

8. Performance Metrics {#performance}

Measuring Impact

```
// Before lazy loading
const measureBefore = {
  initialBundleSize: '5.2MB',
  timeToInteractive: '8.3s',
  firstContentfulPaint: '3.1s',
  numberOfRequests: 145
};

// After lazy loading
const measureAfter = {
  initialBundleSize: '1.8MB', // 65% reduction
  timeToInteractive: '2.4s', // 71% improvement
  firstContentfulPaint: '0.9s', // 71% improvement
  numberOfRequests: 42 // 71% reduction
};
```

Monitoring in Production

```
// Track lazy load performance
const perfObserver = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    if (entry.initiatorType === 'img' || entry.initiatorType === 'script') {
      console.log({
        name: entry.name,
        duration: entry.duration,
        transferSize: entry.transferSize
      });
    }
  }
});

perfObserver.observe({ entryTypes: ['resource'] });
```

9. Common Pitfalls {#pitfalls}

✗ Pitfall 1: Lazy Loading Above-the-Fold Content

```
// BAD: Lazy loading hero image


// GOOD: Load immediately

```

✗ Pitfall 2: Not Setting Dimensions

```
// BAD: Causes layout shift (CLS)


// GOOD: Reserve space

```

❌ Pitfall 3: Memory Leaks with Observers

```
// BAD: Observer not disconnected
const observer = new IntersectionObserver(callback);
images.forEach(img => observer.observe(img));

// GOOD: Unobserve after loading
const observer = new IntersectionObserver((entries, self) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      loadImage(entry.target);
      self.unobserve(entry.target); // ✅ Clean up
    }
  });
});
```

❌ Pitfall 4: Ignoring SEO

```
// BAD: Search engines can't see images


// GOOD: Provide fallback

<noscript>
  
</noscript>
```

❌ Pitfall 5: Over-Engineering

```
// BAD: Unnecessary complexity for simple cases
// Complex custom lazy loading solution for 5 images

// GOOD: Use native lazy loading

```

Summary Cheat Sheet

Quick Decision Tree

```
Need lazy loading?
├─ Few images? → Use native loading="lazy"
├─ Many images? → Use Intersection Observer
├─ Complex logic? → Build custom solution
└─ Legacy browsers? → Use scroll event with throttling
```

Key Takeaways

1. **Always lazy load** images below the fold
2. **Never lazy load** critical above-the-fold content
3. **Use Intersection Observer** for modern implementations
4. **Set explicit dimensions** to prevent layout shifts
5. **Provide placeholders** for better UX
6. **Monitor performance** to measure impact
7. **Handle errors** gracefully
8. **Consider SEO** implications

Performance Checklist

- ☐ Lazy load below-the-fold images
 - ☐ Set width/height on all images
 - ☐ Implement code splitting for large bundles
 - ☐ Use appropriate root margins (50-200px)
 - ☐ Provide loading placeholders
 - ☐ Test on slow network conditions
 - ☐ Monitor Core Web Vitals (LCP, CLS, FID)
 - ☐ Implement error handling
 - ☐ Add noscript fallbacks for SEO
 - ☐ Use CDN for faster delivery
-

Additional Resources

- MDN Web Docs: Intersection Observer API
- web.dev: Lazy loading images and video
- Chrome DevTools: Performance tab for measuring
- Lighthouse: Automated auditing
- WebPageTest: Real-world performance testing